



mali.icd vs mail-arm64.icd

Both `mali.icd` and `mali-arm64.icd` are configuration files used by the OpenCL Installable Client Driver (ICD) Loader. They tell your system (typically Linux on ARM) where to find the proprietary `libmali` library needed to enable GPU-accelerated computing. [ODROID Forum +1](#)

The Core Differences

- `mali.icd`** : The generic, traditional ICD configuration name used by most ARM Linux distributions (like Ubuntu, Debian, or Armbian) on aarch64.
- `mali-arm64.icd`** : A specifically named ICD file often utilized in customized distributions (such as specific Rockchip builds or containerized environments like Frigate) to explicitly designate the 64-bit architecture. [GitHub +4](#)

Contents of the Files

Inside both `.icd` files, you will find a simple text string pointing to the exact location of your `.so` driver file. [OpenSuse +2](#)

For example:

```
/usr/lib/aarch64-linux-gnu/libmali.so
```

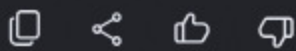
(Note: The exact library name inside the file depends on your specific Mali DDK version, such as `libmali-valhall-g610-g6p0-x11-gbm.so`). [MLC LLM](#)

Which one should you use?

- Use `mali.icd` if:** You are following standard, community-supported setup guides. It is the most universally recognized name by standard OpenCL utilities like `clinfo`. [OpenSuse +2](#)
- Use `mali-arm64.icd` if:** You are working with a vendor-provided or containerized image (e.g., custom Frigate/Rockchip configurations) where the system expects this exact filename to register the 64-bit GPU properly. [GitHub +1](#)

In either case, you can generally verify that your OpenCL stack is working correctly and reading the `.icd` file by installing and running `clinfo`. [OpenSuse +1](#)

For more information on manually integrating ARM Mali blobs, check out the [Setting up OpenCL on RK3588 Guide](#) or dive into the community solutions in the [ODROID Forum](#).



mesa-opengl-icd vs ocl-icd-opengl-dev

Both `mesa-opengl-icd` and `ocl-icd-opengl-dev` are Linux packages used for OpenCL development and execution, but they serve completely different purposes. **One is an actual GPU driver implementation, while the other is a development kit for developers.**

Here is the direct breakdown of how they differ:

Quick Comparison

Feature	mesa-opengl-icd	ocl-icd-opengl-dev
Primary Role	Runtime GPU Driver	Development Kit (Headers & Links)
What it contains	Mesa's open-source OpenCL driver ("Clover" or "Rusticl")	C/C++ header files (<code>cl.h</code>) and development symlinks
When to use it	To run OpenCL apps on open-source drivers (AMD, Intel, Nouveau)	To compile/build your own OpenCL programs from source code
Hardware Target	Open-source PC graphics (not Mali/Rockchip)	Hardware agnostic (works with any GPU/driver)

Deep Dive

1. mesa-opengl-icd (The Driver)

This package is an **Installable Client Driver (ICD)**. It contains the actual compiled code that translates OpenCL commands into instructions your specific graphics card can understand.

- Who it is for:** Users who want to run OpenCL applications (like Blender or Darktable) using open-source drivers.
- How it works:** It provides a `.icd` file that points to Mesa's open-source OpenCL implementations (historically **Clover**, and increasingly the modern Rust-based **Rusticl**).
- ⚠ Important Note for ARM/Mali Users:** If you are working with ARM Mali GPUs (like the Rockchip RK3588 from your previous query), `mesa-opengl-icd` is generally **not** what you want. Mali GPUs require proprietary blobs (`libmali.so`), whereas Mesa primarily supports desktop GPUs like AMD and Intel.

2. ocl-icd-opengl-dev (The Development Kit)

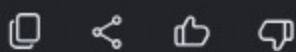
This package does not contain a graphics driver. Instead, it provides the necessary scaffolding for programmers to write and compile OpenCL applications.

- Who it is for:** Developers compiling OpenCL code from scratch.
- What it installs:**
 - Standard OpenCL C headers (e.g., `<CL/cl.h>`).
 - The `libOpenCL.so` development symlink used by compilers (like `gcc` or `clang`) during the build phase.
 - It depends on `ocl-icd-libopencl1` , which is the universal loader that detects your `.icd` files at runtime.

Which one do you need?

- If you are trying to run a pre-compiled OpenCL app on an AMD/Intel PC:** Install `mesa-opengl-icd` .
- If you are building an OpenCL application from source code:** Install `ocl-icd-opengl-dev` .
- If you are setting up an ARM Mali / Rockchip device:** Avoid `mesa-opengl-icd` . Instead, use your proprietary Mali driver (`libmali`) paired with `ocl-icd-libopencl1` (the loader) so your system can see your `mali.icd` configuration.

To help point you in the right direction, what **specific hardware** (e.g., PC or ARM board) and **software/application** are you currently trying to configure?





9 sites



GPU Drivers and SDKs — mlc-llm
0.1.0 documentation

OpenCL SDK™. OpenCL SDK is only required when you want to build your own...



MLC LLM



[HW Accel Support]: RKMPP
AARCH64 · Issue #6914 - GitHub

... /usr/lib/aarch64-linux-gnu/include/armnnTfLiteParser/usr/lib/aarch64-linux-g...



GitHub



zeyadadev/mali-vulkan-icd-wrapper - GitHub

Test it * 2.4 bit testing (default tools): VulkanInfo - Summary: W...